

Toán cơ bản (Basic Math)

Toán học là nền tảng không thể thiếu trong lập trình thi đấu[cite: 1, 2]. Rất nhiều bài toán CP đòi hỏi kiến thức toán cơ bản: từ ước số, UCLN, BCNN, đến số nguyên tố, số học modular, và lũy thừa nhanh[cite: 1, 2]. Đừng lo nếu bạn chưa giỏi toán – bài học này sẽ đi từng bước một, từ khái niệm đến code, với nhiều ví dụ minh họa chi tiết[cite: 1, 2]. Mỗi phần đều có giải thích "tại sao" chứ không chỉ "làm sao"![cite: 1, 2]

1. Ước số (Divisors)

1.1. Ước số là gì?

Ước số của một số nguyên dương n là số nguyên dương d mà n chia hết cho d (tức $n \bmod d = 0$ hay nói cách khác n/d là số nguyên)[cite: 1, 2].

Ví dụ: Các ước số của 12 là: 1, 2, 3, 4, 6, 12[cite: 1, 2].

Ta kiểm tra từng số:[cite: 1, 2]

D	$12 \div D$	CHIA HẾT?	LÀ ƯỚC?
1	12	Có	Ước
2	6	Có	Ước
3	4	Có	Ước
4	3	Có	Ước
5	2.4	Không	Không
6	2	Có	Ước
7	1.71...	Không	Không
12	1	Có	Ước

Vậy 12 có 6 ước: {1, 2, 3, 4, 6, 12}[cite: 1, 2].

1.2. Cách liệt kê ước số "ngây thơ" $O(n)$

Cách đơn giản nhất: thử tất cả số từ 1 đến n [cite: 1, 2].

C++

```
vector<int> divisors;
for (int i = 1; i <= n; i++) {
    if (n % i == 0) {
        divisors.push_back(i);
    }
}
```

Python

```
divisors = []
for i in range(1, n + 1):
    if n % i == 0:
        divisors.append(i)
```

Cách này chạy được khi n nhỏ (ví dụ $n \leq 10^6$) [cite: 1, 2]. Nhưng nếu $n = 10^{12}$ thì sao? Vòng lặp chạy 10^{12} lần – quá chậm! [cite: 1, 2]

1.3. Liệt kê ước số nhanh – $O(\sqrt{n})$

Quan sát cốt lõi: Nếu d là ước của n , thì n/d cũng là ước của n [cite: 1, 2]. Các ước luôn đi thành cặp: $(d, n/d)$ [cite: 1, 2].

Ví dụ với $n = 36$: [cite: 1, 2]

- $1 \times 36 = 36 \rightarrow$ cặp $(1, 36)$ [cite: 1, 2]
- $2 \times 18 = 36 \rightarrow$ cặp $(2, 18)$ [cite: 1, 2]
- $3 \times 12 = 36 \rightarrow$ cặp $(3, 12)$ [cite: 1, 2]
- $4 \times 9 = 36 \rightarrow$ cặp $(4, 9)$ [cite: 1, 2]
- $6 \times 6 = 36 \leftarrow$ cặp $(6, 6)$ ($d = n/d = \sqrt{36}$) [cite: 1, 2]

Nhận xét: trong mỗi cặp $(d, n/d)$, ít nhất một số $\leq \sqrt{n}$ [cite: 1, 2].

Chứng minh: Giả sử phản chứng: cả $d > \sqrt{n}$ và $n/d > \sqrt{n}$ [cite: 1, 2]. Khi đó:

$$d \times (n/d) > \sqrt{n} \times \sqrt{n} = n$$

Nhưng $d \times (n/d) = n$, mâu thuẫn! [cite: 1, 2]

Vì vậy, ta chỉ cần thử d từ 1 đến $\sqrt{n}$ [cite: 1, 2]. Mỗi khi tìm được ước d , ta có luôn ước n/d [cite: 1, 2].

C++

```
vector<long long> divisors;
for (long long i = 1; i * i <= n; i++) {
    if (n % i == 0) {
        divisors.push_back(i);
        if (i != n / i) divisors.push_back(n / i); // tránh trùng khi
            i = sqrt(n)
    }
}
sort(divisors.begin(), divisors.end());
```

Python

```
divisors = []
i = 1
while i * i <= n:
    if n % i == 0:
        divisors.append(i)
        if i != n // i:
            divisors.append(n // i)
    i += 1
divisors.sort()
```

So sánh tốc độ: [cite: 1, 2]

N	$O(N)$ – DUYỆT HẾT	$O(\sqrt{N})$ – DUYỆT CẤP
10^6	10^6 bước	10^3 bước
10^{12}	10^{12} bước (TLE!)	10^6 bước (OK!)

1.4. Đếm số ước

Nếu chỉ cần đếm mà không cần liệt kê:[cite: 1, 2]

C++

```
int count_divisors(long long n) {
    int count = 0;
    for (long long i = 1; i * i <= n; i++) {
        if (n % i == 0) {
            count++;
            if (i != n / i) count++;
        }
    }
    return count;
}
```

Python

```
def count_divisors(n):
    count = 0
    i = 1
    while i * i <= n:
        if n % i == 0:
            count += 1
            if i != n // i:
                count += 1
        i += 1
    return count
```

2. GCD (Ước chung lớn nhất) và LCM (Bội chung nhỏ nhất)

2.1. GCD – Ước chung lớn nhất

$\text{gcd}(a, b)$ là số nguyên dương lớn nhất mà cả a và b đều chia hết[cite: 1, 2].

Ví dụ: $\text{gcd}(12, 18) = 6$: [cite: 1, 2]

- Ước của 12: $\{1, 2, 3, 4, 6, 12\}$ [cite: 1, 2]
- Ước của 18: $\{1, 2, 3, 6, 9, 18\}$ [cite: 1, 2]
- Ước chung: $\{1, 2, 3, 6\}$ [cite: 1, 2]
- Lớn nhất: 6 [cite: 1, 2]

2.2. Thuật toán Euclid – Tìm GCD siêu nhanh

Thay vì liệt kê tất cả ước, ta dùng thuật toán Euclid dựa trên tính chất:[cite: 1, 2]

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

Lặp lại cho đến khi $b = 0$, khi đó $\gcd(a, 0) = a$ [cite: 1, 2].

Chạy tay: $\gcd(48, 18)$ [cite: 1, 2]

- Bước 1: $\gcd(48, 18) \rightarrow 48 \bmod 18 = 12 \rightarrow \gcd(18, 12)$ [cite: 1, 2]
- Bước 2: $\gcd(18, 12) \rightarrow 18 \bmod 12 = 6 \rightarrow \gcd(12, 6)$ [cite: 1, 2]
- Bước 3: $\gcd(12, 6) \rightarrow 12 \bmod 6 = 0 \rightarrow \gcd(6, 0)$ [cite: 1, 2]
- Bước 4: $\gcd(6, 0) \rightarrow b = 0$ trả về $a = 6$ [cite: 1, 2]

Kết quả: $\gcd(48, 18) = 6$. [cite: 1, 2]

Code

C++ (recursive)

```
int gcd(int a, int b) {
    return b == 0 ? a : gcd(b, a % b);
}
```

C++ (iterative)

```
int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
```

Python

```
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a
```

Trong C++17 trở lên, có thể dùng `std::gcd` trong `<numeric>`[cite: 1, 2].

Độ phức tạp: $O(\log(\min(a, b)))$ [cite: 1, 2].

2.3. Ví dụ khác: $\gcd(252, 105)$

- $\gcd(252, 105)$: $252 \bmod 105 = 42 \rightarrow \gcd(105, 42)$ [cite: 1, 2]
- $\gcd(105, 42)$: $105 \bmod 42 = 21 \rightarrow \gcd(42, 21)$ [cite: 1, 2]
- $\gcd(42, 21)$: $42 \bmod 21 = 0 \rightarrow \gcd(21, 0) = 21$ [cite: 1, 2]

Kết quả: $\gcd(252, 105) = 21$. [cite: 1, 2]

2.4. LCM – Bội chung nhỏ nhất

$\text{lcm}(a, b)$ là số nguyên dương nhỏ nhất mà cả a và b đều là ước[cite: 1, 2].

Công thức:[cite: 1, 2]

$$\text{lcm}(a, b) = \frac{a \times b}{\text{gcd}(a, b)}$$

Ví dụ:[cite: 1, 2]

$$\text{lcm}(12, 18) = \frac{12 \times 18}{6} = 36$$

Code – chú ý tránh tràn số[cite: 1, 2]

C++

```
long long lcm(long long a, long long b) {  
    return a / gcd(a, b) * b; // chia truoc, nhan sau  
}  
  
// SAI - co the tran so  
long long lcm_wrong(long long a, long long b) {  
    return a * b / gcd(a, b);  
}
```

Python

```
def lcm(a, b):  
    return a // gcd(a, b) * b
```

3. Số nguyên tố (Prime Numbers)

3.1. Số nguyên tố là gì?

Số nguyên tố là số nguyên > 1 mà chỉ có đúng 2 ước: 1 và chính nó[cite: 1, 2].

Các số nguyên tố đầu tiên: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, ... [cite: 1, 2]

Lưu ý:[cite: 1, 2]

- 1 không phải số nguyên tố[cite: 1, 2].
- 2 là số nguyên tố chẵn duy nhất[cite: 1, 2].

3.2. Kiểm tra số nguyên tố – $O(\sqrt{n})$

C++

```
bool is_prime(long long n) {  
    if (n < 2) return false;  
    for (long long i = 2; i * i <= n; i++) {  
        if (n % i == 0) return false;  
    }  
}
```

```

    return true;
}

```

Python

```

def is_prime(n):
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True

```

3.3. Phiên bản tối ưu hơn

C++

```

bool is_prime(long long n) {
    if (n < 2) return false;
    if (n < 4) return true; // 2, 3
    if (n % 2 == 0) return false;
    if (n % 3 == 0) return false;
    for (long long i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) return false;
    }
    return true;
}

```

Python

```

def is_prime(n):
    if n < 2:
        return False
    if n < 4:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True

```

Giải thích: Mọi số nguyên tố lớn hơn 3 đều có dạng $6k \pm 1$ [cite: 1, 2].

3.4. Ví dụ kiểm tra

`is_prime(17)`? [cite: 1, 2]

- $i = 2$: không chia hết[cite: 1, 2]
- $i = 3$: không chia hết[cite: 1, 2]
- $i = 4$: không chia hết[cite: 1, 2]
- $i = 5$: $5 \times 5 > 17 \rightarrow$ dừng[cite: 1, 2]

$\rightarrow 17$ là số nguyên tố[cite: 1, 2].

`is_prime(15)?`[cite: 1, 2]

- $i = 2$: không chia hết[cite: 1, 2]
- $i = 3$: $15 \bmod 3 = 0 \rightarrow$ dừng[cite: 1, 2]

$\rightarrow 15$ không phải số nguyên tố[cite: 1, 2].

4. Sàng Eratosthenes – Tìm tất cả số nguyên tố đến N

4.1. Ý tưởng

Tìm tất cả số nguyên tố đến N trong $O(N \log \log N)$ [cite: 1, 2].

4.2. Thuật toán

1. Tạo mảng `is_prime[0..N]`, ban đầu gán `true`[cite: 1, 2].
2. Gán `is_prime[0] = is_prime[1] = false`[cite: 1, 2].
3. Duyệt i từ 2 đến $\sqrt{N}$ [cite: 1, 2].
4. Nếu `is_prime[i] = true`, đánh dấu các bội của i từ i^2 là `false`[cite: 1, 2].

4.3. Minh họa với $N = 30$

Ban đầu, coi mọi số từ 2 đến 30 là nguyên tố[cite: 1, 2].

- Bước $i = 2$: loại 4, 6, 8, 10, $\dots$, 30[cite: 1, 2]
- Bước $i = 3$: loại thêm 9, 15, 21, 27[cite: 1, 2]
- Bước $i = 4$: đã bị loại, bỏ qua[cite: 1, 2]
- Bước $i = 5$: loại thêm 25[cite: 1, 2]

Kết quả: $\{2, 3, 5, 7, 11, 13, 17, 19, 23, 29\}$ [cite: 1, 2].

4.4. Code

C++

```
const int MAXN = 100000000;
bool is_prime[MAXN + 1];

void sieve() {
    fill(is_prime, is_prime + MAXN + 1, true);
    is_prime[0] = is_prime[1] = false;
    for (int i = 2; 1LL * i * i <= MAXN; i++) {
        if (is_prime[i]) {
            for (int j = i * i; j <= MAXN; j += i) {
                is_prime[j] = false;
            }
        }
    }
}
```

Python

```
def sieve(n):
    is_prime = [True] * (n + 1)
    if n >= 0:
        is_prime[0] = False
    if n >= 1:
        is_prime[1] = False
    i = 2
    while i * i <= n:
        if is_prime[i]:
            j = i * i
            while j <= n:
                is_prime[j] = False
                j += i
        i += 1
    return is_prime
```

Tại sao bắt đầu từ i^2 ? Vì các bội nhỏ hơn i^2 ($i \times 2, i \times 3, \dots, i \times (i - 1)$) đã bị loại bởi các số nhỏ hơn trước đó rồi[cite: 1, 2].

5. Số học Modular (Modular Arithmetic)

5.1. Tại sao cần modular?

Trong CP, đáp án có thể rất lớn, nên đề thường yêu cầu in kết quả theo modulo $10^9 + 7$ [cite: 1, 2].

$$10^9 + 7 = 1000000007$$

Đây là một số nguyên tố rất hay dùng[cite: 1, 2].

5.2. Các tính chất cơ bản

- $(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$ [cite: 1, 2]
- $(a \times b) \bmod m = ((a \bmod m) \times (b \bmod m)) \bmod m$ [cite: 1, 2]
- $(a - b) \bmod m = ((a \bmod m) - (b \bmod m) + m) \bmod m$ [cite: 1, 2]

Lưu ý: Phép chia không áp dụng kiểu này một cách trực tiếp[cite: 1, 2].

5.3. Code

C++

```
const int MOD = 1e9 + 7;

int add(int a, int b) {
    return ((long long)a + b) % MOD;
}

int mul(int a, int b) {
    return (1LL * a * b) % MOD;
}

int sub(int a, int b) {
    return ((long long)a - b + MOD) % MOD;
}
```

Python

```
MOD = 10**9 + 7

def add(a, b):
    return (a + b) % MOD

def mul(a, b):
    return (a * b) % MOD

def sub(a, b):
    return (a - b + MOD) % MOD
```

Lỗi phổ biến: quên ép long long trong C++ khi nhân hai số lớn[cite: 1, 2].

6. Lũy thừa nhanh (Fast Exponentiation)

6.1. Bài toán

Tính $a^b \bmod m$ với b rất lớn, có thể tới 10^{18} [cite: 1, 2].

Cách nhân a với chính nó b lần là $O(b)$, quá chậm[cite: 1, 2].

6.2. Ý tưởng

- Nếu b chẵn: $a^b = (a^{b/2})^2$ [cite: 1, 2]
- Nếu b lẻ: $a^b = a \times a^{b-1}$ [cite: 1, 2]

Mỗi bước giảm số mũ đi khoảng một nửa, nên tổng thời gian là $O(\log b)$ [cite: 1, 2].

6.3. Chạy tay: $3^{13} \bmod 100$

Ta có:[cite: 1, 2]

$$13 = (1101)_2 = 8 + 4 + 1$$

Nên:[cite: 1, 2]

$$3^{13} = 3^8 \times 3^4 \times 3^1$$

Tính lần lượt:[cite: 1, 2]

- $3^1 \bmod 100 = 3$ [cite: 1, 2]
- $3^2 \bmod 100 = 9$ [cite: 1, 2]
- $3^4 \bmod 100 = 81$ [cite: 1, 2]
- $3^8 \bmod 100 = 61$ [cite: 1, 2]

Rồi nhân các phần cần thiết:[cite: 1, 2]

$$3 \times 81 \times 61 \bmod 100 = 23$$

Vậy $3^{13} \bmod 100 = 23$ [cite: 1, 2].

6.4. Code

C++

```
long long power(long long a, long long b, long long m) {
    long long res = 1;
    a %= m;
    while (b > 0) {
        if (b % 2 == 1) {
            res = (res * a) % m;
        }
        a = (a * a) % m;
        b /= 2;
    }
    return res;
}
```

Python

```
def power(a, b, m):
    res = 1
    a %= m
    while b > 0:
        if b % 2 == 1:
            res = (res * a) % m
        a = (a * a) % m
        b //= 2
    return res
```

6.5. Ví dụ sử dụng

C++

```
cout << power(2, 100, 1000000007) << endl; // 976371285
cout << power(3, 1000000, 1000000007) << endl;
```

Python

```
print(power(2, 100, 1000000007)) # 976371285
print(power(3, 1000000, 1000000007))
```

7. Tổng hợp: Các lỗi phổ biến

LỖI	HẬU QUẢ	CÁCH TRÁNH
Duyệt ước đến n thay vì $\sqrt{n}$	TLE khi n lớn	Dùng $i \times i \leq n$
LCM: nhân trước chia sau	Tràn long long	$a / \gcd(a, b) \times b$
Quên 1 không phải nguyên tố	WA	Kiểm tra $n < 2$
Sàng: bắt đầu từ $2i$ thay vì i^2	Chậm hơn	Bắt đầu từ $i \times i$
Modular: quên ép long long khi nhân	Tràn int	Dùng <code>1LL * a * b</code>
Modular: quên cộng MOD khi trừ	Kết quả âm	$(a - b + \text{MOD}) \bmod \text{MOD}$
Lũy thừa: dùng vòng lặp $O(b)$	TLE	Dùng binary exponentiation

Tóm tắt bài học

KHÁI NIỆM	KỸ THUẬT CHÍNH	ĐỘ PHỨC TẠP
Liệt kê ước số	Duyệt đến $\sqrt{n}$, thêm cặp	$O(\sqrt{n})$
GCD	Thuật toán Euclid	$O(\log(\min(a, b)))$
LCM	$a / \gcd(a, b) \times b$	$O(\log(\min(a, b)))$
Kiểm tra nguyên tố	Duyệt đến $\sqrt{n}$	$O(\sqrt{n})$
Sàng nguyên tố	Sàng Eratosthenes	$O(N \log \log N)$
Modular arithmetic	Mod sau mỗi phép tính	$O(1)$ mỗi phép
Lũy thừa nhanh	Bình phương liên tiếp	$O(\log b)$

Bây giờ hãy thử sức với các bài tập[cite: 1, 2]. Bắt đầu từ các bài về ước số và GCD, rồi tiến đến bài modular và lũy thừa nhanh nhé![cite: 1, 2]

Bài tập

- Tìm UCLN, BCNN (0/10)[cite: 1, 2]
- Số nguyên tố (0/10)[cite: 1, 2]
- Ước số chung bản nâng cao (0/15)[cite: 1, 2]
- Ước số chung lớn nhất (Khó) (0/15)[cite: 1, 2]
- CSES - Trailing Zeros | Số không ở cuối (0/15)[cite: 1, 2]
- CSES - Bit Strings | Xâu nhị phân (0/15)[cite: 1, 2]
- Số lượng ước số của n (0/20)[cite: 1, 2]
- KT Số nguyên tố (0/20)[cite: 1, 2]
- CSES - Exponentiation | Lũy thừa (0/20)[cite: 1, 2]
- CSES - Two Knights | Hai quân mã (0/20)[cite: 1, 2]
- CSES - Counting Divisors | Đếm ước (0/25)[cite: 1, 2]